NoSQL Systems

Introduction to MapReduce

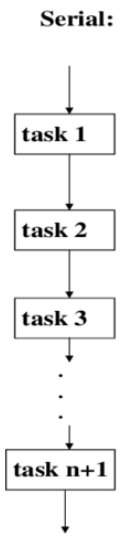
Vinu Venugopal
ScaDS.ai Lab, IIIT Bangalore

Introduction

- Traditional programming is **serial**

Unix pipeline

```
cat log.txt | grep "ERROR" | sort | uniq -c | sort -nr
```



uniq -c – counts identical lines.
sort -nr – sorts counts in descending order.

Introduction

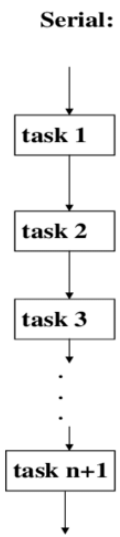
- Traditional programming is **serial**

Unix pipeline

```
cat log.txt | grep "ERROR" | sort | uniq -c | sort -nr
```

Conceptually:

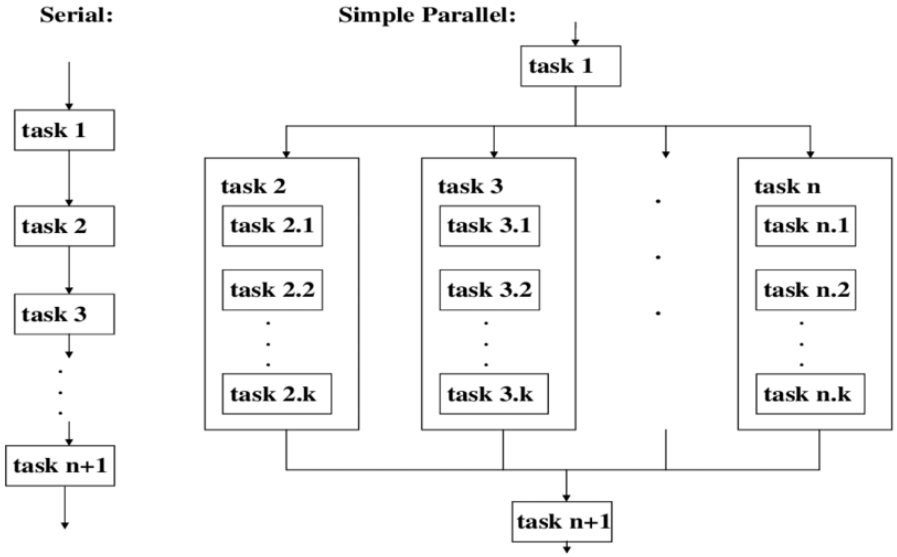
Process A → stdout → pipe → stdin → Process B → pipe → Process C



tells OS to connect the stdout of the cmd on the left to the stdin of the cmd on the right – OS managed buffer in kernel, providing IPC

Introduction

- Traditional programming is **serial**
- Parallel programming
  - **Break processing** into parts that can be **executed concurrently on multiple processors**
- Challenge
  - Identify **tasks** that can run concurrently and/or **groups of data** that can be processed **concurrently**
  - **Not all problems** can be parallelized



Simplest environment for parallel processing

- **No dependency** among data
- Data can be **split into lots of smaller chunks** - shards
- Each **process** can **work on a chunk**

MapReduce

- Created by **Google** in 2004
  - **Jeffrey** Dean and **Sanjay** Ghemawat

MapReduce

- Created by **Google** in 2004
  - **Jeffrey** Dean and **Sanjay** Ghemawat
- Inspired by LISP
  - **Map(function, set of values)**
    - Applies function to each value in the set

      e.g., map (length(), [“”, “a”, “ab”, “abc”]) ⇒ (0 1 2 3)
  - **Reduce(function, set of values)**
    - Combines all the values using a binary function

      e.g., reduce (sum(), (1,2,3,4,5)) ⇒ 15

MapReduce

- **Framework** for parallel computing
- Programmers get simple **API**
- Don’t have to worry about handling
  - Parallelization
  - Data distribution
  - Load balancing
  - Fault tolerance
- Allows one to **process huge amounts of data** (terabytes and petabytes) on **thousands of processors**

Implementations

- Google
  - Original **proprietary implementation**
- Apache **Hadoop MapReduce**
  - Most common (open-source) implementation
  - Built to specs defined by Google
- Amazon Elastic MapReduce
  - Uses Hadoop MapReduce running on Amazon EC2
- Other Cloud Solutions
  - Microsoft Azure HDInsight
  - Google Cloud MapReduce for App Engine

MapReduce was invented by Julius Caesar

Roman military strategist and politician, known for famous quotes: "veni, vidi, vici”, "divide et impera” (latin)

Source: https://www.fiware.org/2014/05/30/mapreduce-was-really-invented-by-julius-caesar/

MapReduce was invented by Julius Caesar

So, how does it work? Or better said, how would Julius Caesar **solve a Big Data problem using divide and conquer**?

- Caesar arrives Alexandria after defeating the Egyptian army; enters the Ancient Library of Alexandria.
- As a great historian and writer, He wants to know how many pages of the books are written in Latin.
- To gauge how deeply Latin culture is rooted in Egypt.

MapReduce was invented by Julius Caesar

MapReduce was invented by Julius Caesar

Caesar arranges a **Centuria** (a set of 80 soldiers), instructs each member to inspect a batch of books and to report to their **Centurion** (the official in charge of the Centuria) the number of pages they found in a book that is written in Latin.

MapReduce

Map

- Grab the relevant data from the source
- User function gets called for each chunk of input
- Spits out (key, value) pairs

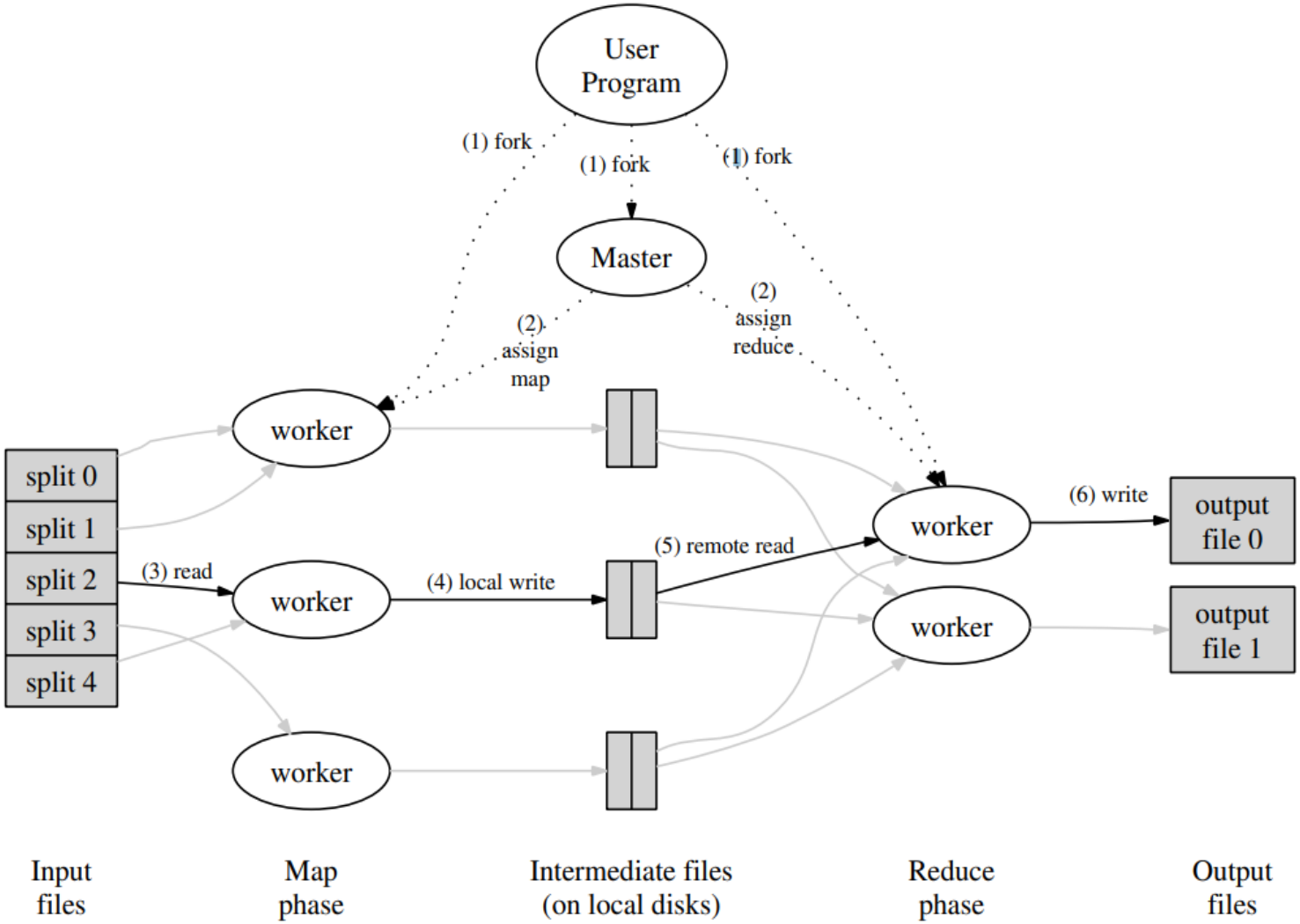
Reduce

- Aggregate the results
- User function gets called **for each unique key** with **all values corresponding to that key**

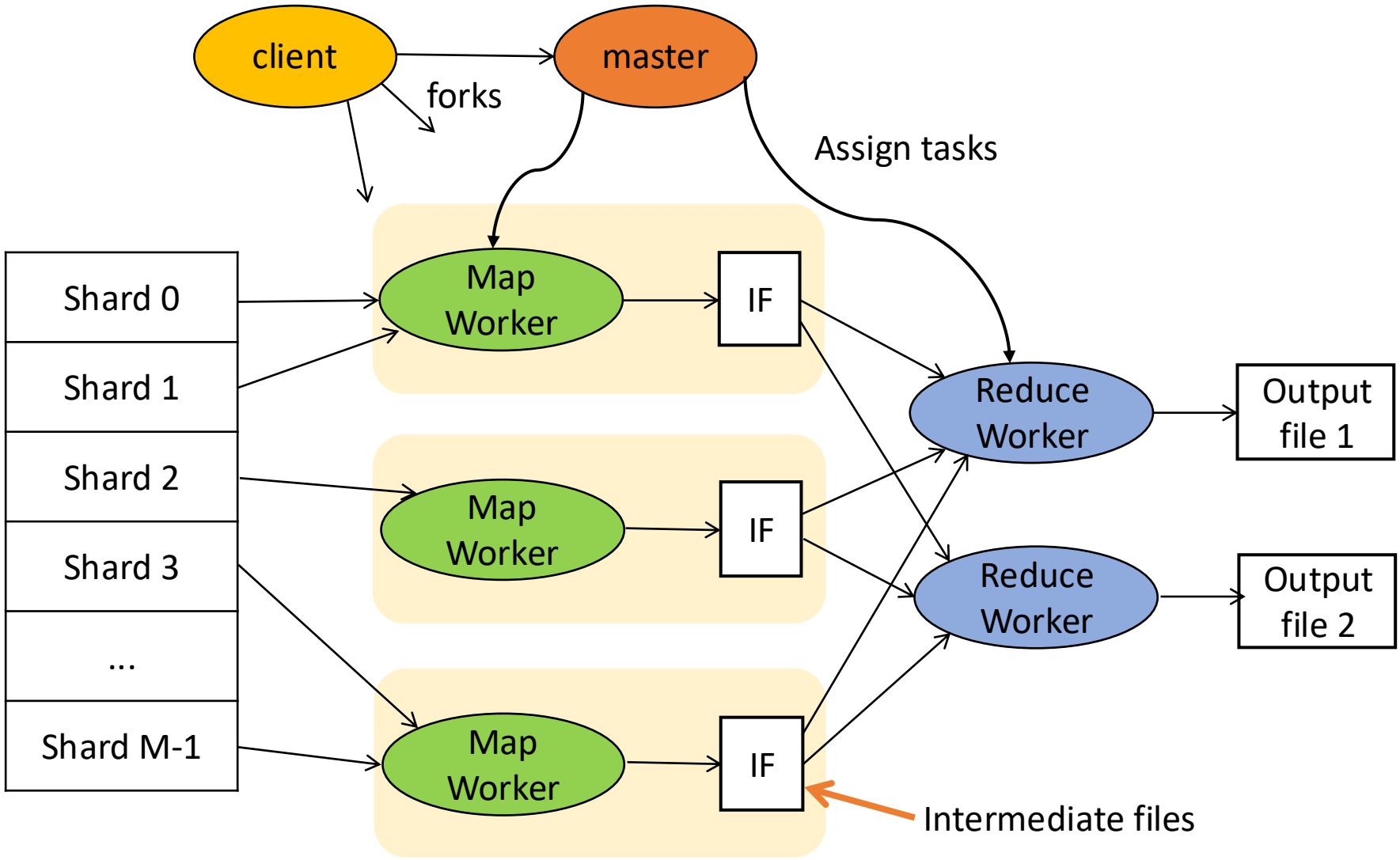
MapReduce

- Automatically partition input data into M **shards**
- **Map**: (input shard) → intermediate(key/value pairs)
  - Discard unnecessary data and generate (key, value) sets
- Framework **groups together all intermediate values** with the same intermediate key & pass them to the Reduce function
- **Reduce**: intermediate(key/set of values) → result files
  - Input: key & set of values
  - Merge these values together to form a smaller set of values
- Intermediate results from Map are divided into R pieces corresponding to R reducers using a partitioning function (e.g., hash(key) mod R)
- The user specifies the # of partitions (R) and the partitioning function

MapReduce: The Big Picture



MapReduce: The Big Picture



MapReduce: Steps Involved

Map Worker

Map

- Grab the relevant data from the source (parse into key, value)
- Write it to an intermediate file

Partition

- Partitioning: identify which of R reducers will handle which keys
- Map partitions data to target it to one of R Reduce workers based on a partitioning function (both R and partitioning function are user defined)

Reduce Worker

Shuffle & Sort

- Shuffle: Fetch the relevant partition of the output from all mappers
- Sort by keys (different mappers may have sent data with the same key)

Reduce

- Input is the sorted output of mappers
- Call the user Reduce function per key with the list of values for that key to aggregate the results

Step 1: Split input files into chunks (shards)

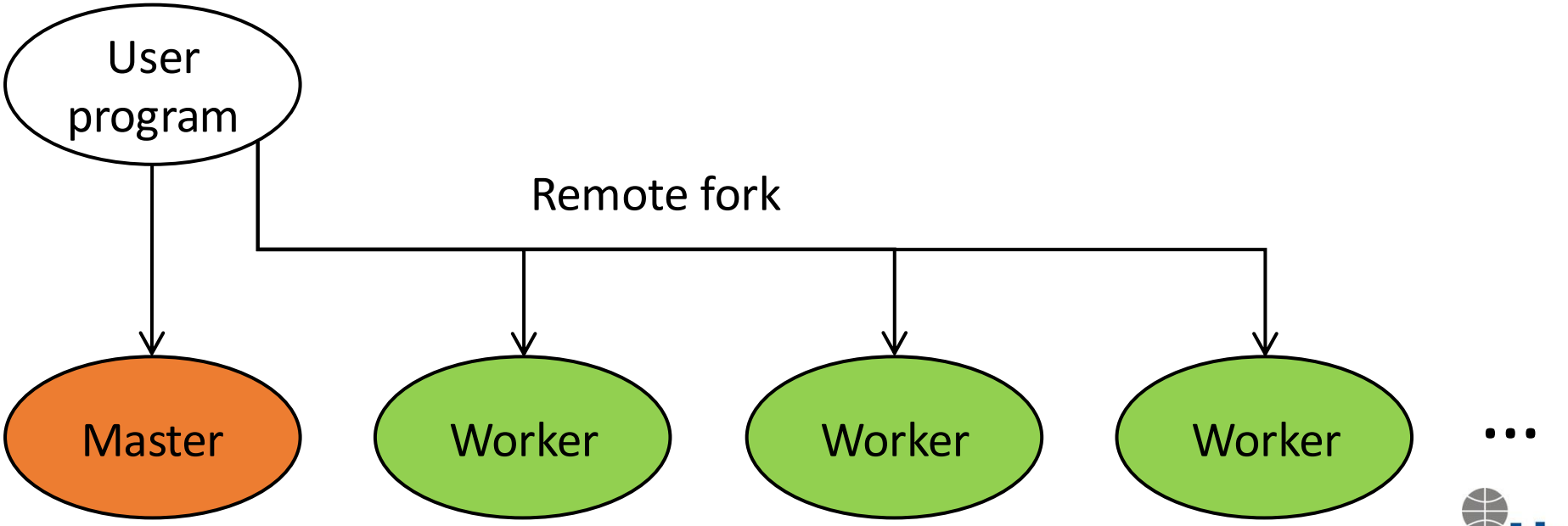
Shard 0	Shard 1	Shard 2	Shard 3	…	Shard M-1

Input Files

Divide into ***M* shards**

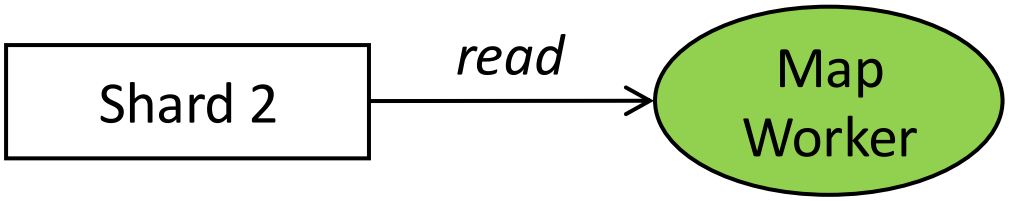
Step 2: Fork processes

- Start up many copies of the program on a cluster of machines
  - **One master**: scheduler & coordinator
  - Lots of **workers**
- Idle workers are assigned either:
  - **map tasks** (each works on a shard) – there are ***M*** map tasks
  - **reduce tasks** (each works on intermediate files) – there are ***R***
    - *R* = # partitions, defined by the user



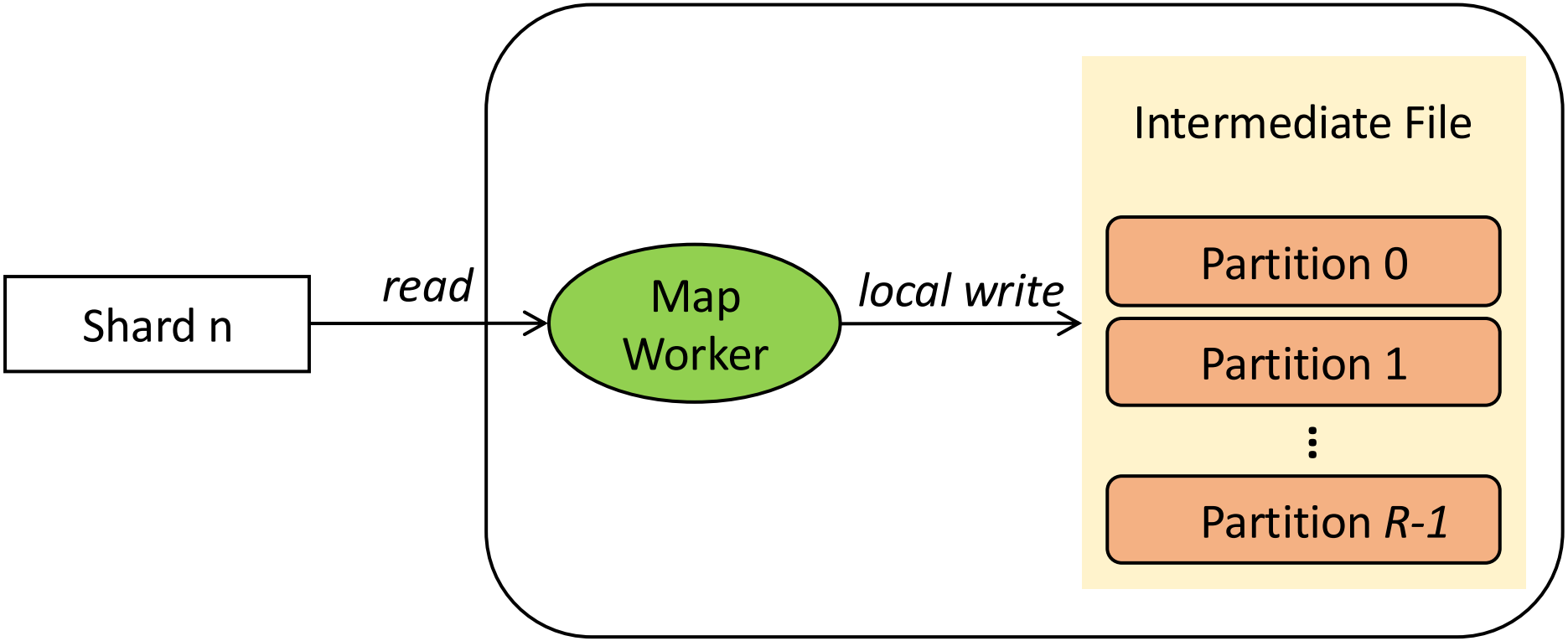
Step 3: Run Map Tasks

- **Reads** contents of the input shard assigned to it
- **Parses key/value pairs** out of the input data
- **Passes** each pair to a user-defined ***map*** function – Produces intermediate key/value pairs
  - These are **buffered in memory**



Step 4: Create intermediate files

- **Intermediate key/value pairs** produced by the user's ***map*** function buffered in memory and **are periodically written to the local disk**
  - Partitioned into ***R*** regions by a partitioning function

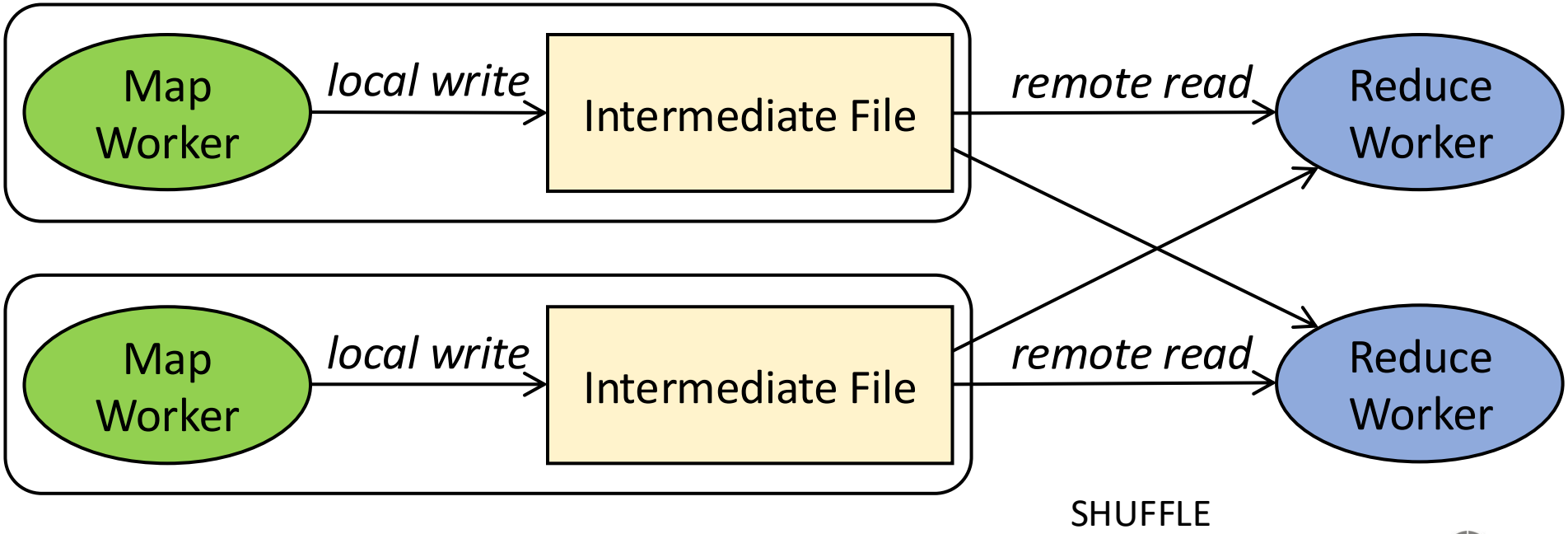


Step 4a: Partitioning

- Map data would be **eventually processed by the reduce workers**
- **Partition function**
  - Decides **which of *R* reduce workers will work on which key**
    - Default function: ***hash(key) mod R***
    - Therefore, Map worker partitions the data by keys
- Each Reduce worker will **later read their partition** from every Map worker

Step 5: Reduce Task: Sorting

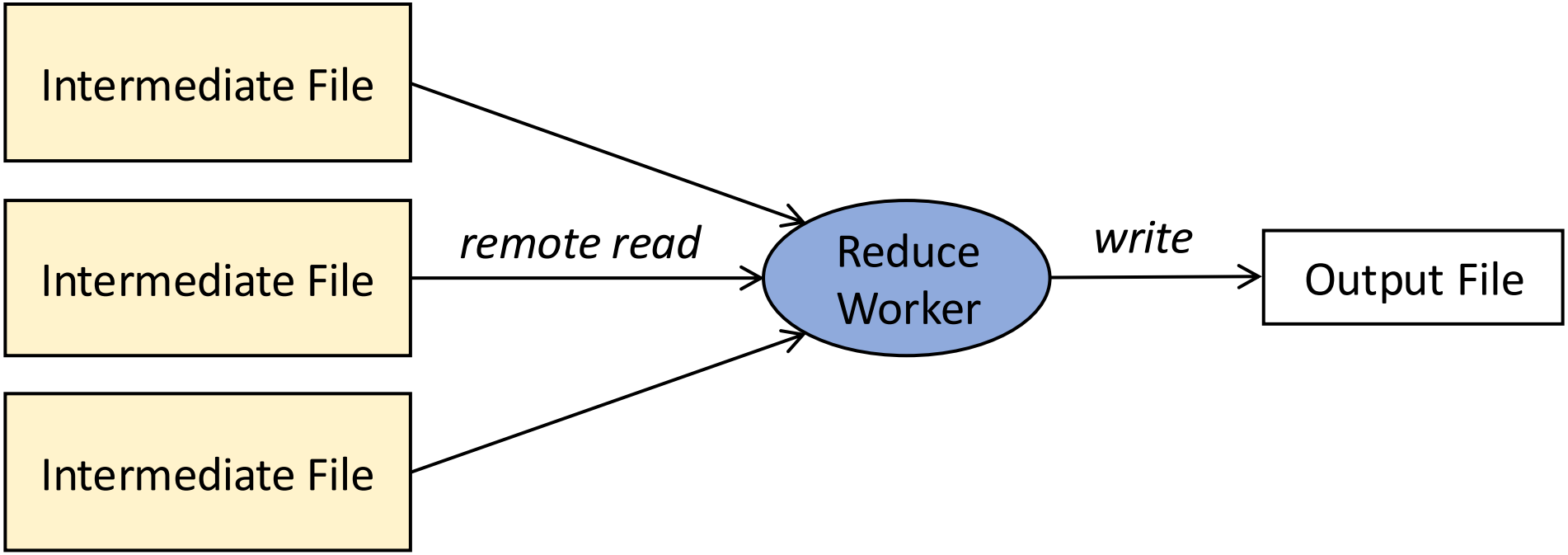
- Reduce worker **gets notified by the master** about the location of intermediate files for its partition
- **Shuffle**: Uses RPCs to read the data from the local disks of the map workers
- **Sort**: When the ***reduce*** worker reads intermediate data for its partition – It sorts the data by the intermediate keys
  - All occurrences of the same key are grouped together



Step 6: Reduce Task: Reduce

- The sort phase grouped data with a unique intermediate key
- User's ***Reduce*** function is given the key and the set of intermediate values for that key

  < key, (value1, value2, value3, value4, ...) >
- The output of the *Reduce* function is appended to an output file



Step 7: Return to user

- When all ***map*** and ***reduce*** tasks have completed, the master wakes up the user program
- The ***MapReduce*** returns the control to the user program and the program can resume execution.
  - Output of ***MapReduce*** is available in ***R*** output files

Some Important Point!

- In a MR job, reducers do not start executing the **reduce()** method until all Map jobs are completed. But, Reducers start copying the output data from the mappers as soon as they are available.
- All reducers store their output part-r-xxxxx files in the output directory specified in the mapreduce job instead of in local file system.
- Map tasks output files are not stored on HDFS, but are stored on local file system
- We **cannot** set the number of map tasks in a mapreduce job. (But this can be set at site level with mapreduce.job.maps configuration property in mapred-site.xml file.)
- We can set the no of reduce tasks of our choice. (Reduce tasks can be setup at site level as well with mapreduce.job.reduces configuration property in mapred-site.xml file.)
- Reducers cannot talk to each other – they run in isolation.
- Hadoop is not a hardware aware system.

Example

Count **# occurrences of each word** in a collection of documents

- **Map**:
  - Parse data; output each word and a count (1)
- **Reduce**:
  - Sort: sort by keys (words)
  - Reduce: Sum together counts of each key (word)

Example

Count **# occurrences of each word** in a collection of documents

- **Map**:
  - Parse data; output each word and a count (1)
- **Reduce**:
  - Sort: sort by keys (words)
  - Reduce: Sum together counts of each key (word)

```
map(String key, String value):
// key: document name, value: document contents
for each word w in value:
   EmitIntermediate(w, "1");

reduce(String key, Iterator values):
// key: a word; values: a list of counts
int result = 0;
for each v in values:
   result += ParseInt(v);
Emit(AsString(result));
```

Example

It will be seen that this mere painstaking burrower and grub-worm of a poor devil of a Sub-Sub appears to have gone through the long Vaticans and street-stalls of the earth, picking up whatever random allusions to whales he could anyways find in any book whatsoever, sacred or profane. Therefore you must not, in every case at least, take the higgledy-piggledy whale statements, however authentic, in these extracts, for veritable gospel cetology. Far from it. As touching the ancient authors generally, as well as the poets here appearing, these extracts are solely valuable or entertaining, as affording a glancing bird's eye view of what has been promiscuously said, thought, fancied, and sung of Leviathan, by many nations and generations, including our own.

MAP

it 1
will 1
be 1
seen 1
that 1
this 1
mere 1
painstaking 1
burrower 1
and 1
grub-worm 1
of 1
a 1
poor 1
devil 1
of 1
a 1
sub-sub 1
appears 1
to 1
have 1
gone 1

REDUCE

…
a 1
a 1
aback 1
aback 1
abaft 1
abaft 1
abandon 1
abandon 1
abandon 1
abandoned 1
abandoned 1
abandoned 1
abandoned 1
abandoned 1
abandoned 1
abandoned 1
abandoned 1
abandonedly 1
abandonment 1
abandonment 1
abased 1
abased 1

a 4736
aback 2
abaft 2
abandon 3
abandoned 7
abandonedly 1
abandonment 2
abased 2
abasement 1
abashed 2
abate 1
abated 3
abatement 1
abating 2
abbreviate 1
abbreviation 1
abeam 1
abed 2
abednego 1
abel 1
abhorred 3
abhorrence 1

Running your first MapReduce program in Hadoop

- The class WordCount.java is available on Moodle for download.
- It requires the basic Hadoop jars for compiling which can be obtained from

https://hadoop.apache.org/releases.html

Running your first MapReduce program in Hadoop

- Compile and copy the WordCount examples into a single Jar file (e.g.: WordCount.jar, which is also available on Moodle).
- First download and unpack the input articles from Wikipedia-50-ARTICLES.tar.gz into your regular UNIX directory
  - (and only then copy them into your HDFS directory via: hadoop dfs –copyFromLocal ...)
- Run the MapReduce job via:

  hadoop jar WordCount.jar WordCount <your_input_directory> <your_output_directory>

Note: <your_output_directory> must be a new directory that may not yet exist already!

Limitations of MapReduce

- Not all problems could be implemented as a MR program.
- Batch nature and lack of real-time guarantees => Not good for **Real-time** processing.
- MR is not a good fit for problems with **global state information** or frequent **random data access**:
  - Many machine learning or graph algorithms require maintenance of centralized information and this implies a single task.
  - Many database applications need frequent random access to individual data objects (e.g., tuples).

➢ Use the right tool for the right job!